

Assignment - 02

Tutorial-01

- 1) The concept of storing multiple data types in a contiguous block of memory is known as Array.
- 2) Explain the role of a compiler in the context of programming languages.

A compiler is a software tool that translates human-readable source code written in a high-level programming language into machine code or an intermediate code that can be executed by computer.

1) translation:

Convert source code written in a high-level programming language into machine code or an intermediate code.

2) Optimization:

Analyzer and restructures the code to improve its efficiency, making the resulting program run faster and use fewer resources.

3) Error checking:

Identifier and reports syntax and semantic errors in the source code, helping programmer detect and fix issues before execution.

4) Code Generation:

Produces executable code that can be directly executed by the computer hardware or an interpreter.

5) Linking:

Combines separately compiled modules or libraries into a single executable program, resolving references between them.

3) What is the purpose of storage classes in C programming, and provide examples of each?

In C programming, storage classes determine the scope, lifetime, and visibility of variables. There are four storage classes in C.

1) auto:

Purpose: used to declare local variables that are local to a block and have automatic storage duration.

Eg:

```
void exampleFunction() {
    auto int localVar = 10;
}
```

2. Register:

Purpose: Similar to auto, but suggests the compiler to store the variable in register for faster access.

eg:

```
void exampleFunction() {
    register int regVar = 20;
}
```

3. Static:

Purpose: used for variables with a longer lifetime. It retains its value b/w function calls and has file scope. It is declared outside any function.

eg:

```
void exampleFunction() {
    static int staticVar = 30;
}
```

4. extern:

- Purpose: used to declare variables that are defined in another source file or in the global scope.

Eg:-

```
int globalvar = 40;  
extern int globalvar;
```

4) What is the Function of an Assembler in the compilation process?

- Converts high-level code to machine code
- Translates assembly code to machine code
- Links object files
- Loads the program into memory

5) Assembly language is a machine-independent and can be executed on any computer. (True or False)

Tutorial - 02

1) Describe the purpose of the 'void' data type in a programming and provide an example of its use.

In a programming, the 'void' data type is used to indicate that a function does not return any value. It is also used as a placeholder for an empty or unspecified type in certain contexts.

2) Return Type: 'void' is used to declare function that do not return a value

eg:- #include <stdio.h>

```
void greet() {
```

```
    printf("Hello, world!\n");
```

```
}
```

```
int main()
```

```
    greet();
```

```
    return 0;
```

```
}
```

3. As a pointer type: 'void' pointer (or generic pointer) are used when you want to create a pointer that can point to data of any type.

```
eg:- # include <stdio.h>
int main()
{
    int intvalue = 42;
    float floatvalue = 3.14;
    void* generic pointer;
    generic pointer = &intvalue;
```

```
printf("value of generic pointer: %d\n",
    *(int*)generic pointer);
```

```
generic pointer = &floatvalue;
```

```
printf("value pointed by generic pointer
    is %f\n", *(float*)generic pointer);
```

```
return 0;
```

```
}
```

Q2) A Program is a set of instructions, while a process is a program in execution.

Q3) Discuss the primary purpose of using logical operators in C programming.

In C programming, logical operators are used to perform logical operations on boolean values (True or False).

The primary purpose of logical operators is to manipulate and

evaluate condition in control flow statements, such as if statements and loops. They allow you to combine or modify conditions to make decisions about the flow of a program based on certain criteria.

Q1)

The three main logical operators in C are:

1. logical AND ('&'): Returns true if both operands are true.
2. logical OR ('|'): Returns true if at least one of the operands is true.
3. logical NOT ('!'): Returns true if the operand is false, and false if the operand is true.

Q4) Arithmetic operators in C include addition, subtraction, multiplication, and division.

(True or False)

Q5) Which qualifier is used to declare an integer variable that can hold larger values? ~~add~~

- a) long
- b) short
- c) int
- d) void

THORAL:- 03

- Q1) Define the terms "Function definition", "Function declaration", and "Function call" in the context of programming.

Function Definition:

Definition: A function definition specifies the behavior of a function, including the code that will be executed when the function is called.

• **Key Point:** It outlines the implementation details of a function.

Function Declaration:

• **Declaration:** A function declaration tells the compiler or interpreter about the existence of a function, including its name, parameters, and return type.

• **Key Point:** It provides a signature for the function without specifying the actual implementation.

3. Function Call:

• **Call:** A Function call is the act of invoking or executing a function, causing the code within the function definition to be executed.

• **Key Point:** It triggers the execution of the specified function at a particular point in the program.

Q2.) Explain the concept of recursion in function and provide an example.

Recursion is a programming concept where a function calls itself in order to solve a problem. In a recursive function, the solution to a problem is expressed in terms of smaller instances of the same problem.

Eg.: to calculate factorial of a number.

```
#include <stdio.h>
int Factorial (int n) {
    if (n == 0 || n == 1) {
        return 1;
    } else {
        return n * Factorial (n-1);
    }
}
```

}

}

int main ()

int number = 5;

int result = factorial (number);

printf ("factorial of %d is %d\n", number, result);

return 0;

}

In this C program, The base check is if the number is 0 or 1, in which case the factorial is 1, otherwise, the function calls itself with the argument (n-1), until the base case is reached. The 'main' function then demonstrates the usage of the 'factorial' function for the number n.

3) The "&" operator in C is used for:

a) Address of a variable

b) Value at memory address

c) Multiplication

d) Division

4) A one-dimensional array is also known as a matrix. (True or False)

5) To declare a pointer in C, you use the _____ symbol.

Tutorial :- 04

- 1) Provide an Example of a situation where using a one-dimensional array would be appropriate.

A one-dimensional array is appropriate when you need to represent a collection of elements in a linear or sequential manner.

eg:-

In C, you can use one-dimensional array to store daily temperature in a similar manner.

```
#include <stdio.h>
```

```
int main() {
```

```
int weekly_temperature[] = {78, 82, 79, 81, 88, 91, 87};
```

```
int Wednesday_temperature = weekly_temperature[2];
```

```
printf("Wednesday temperature: %d degree Fahrenheit\n", Wednesday_temperature);
```

```
return 0;
```

```
}
```

Q1) How are arrays in C stored in memory, and how does this impact their performance?

In C, arrays are contiguous blocks of memory where elements of the same type are stored sequentially. The memory layout of an array is such that each element is stored at a location immediately following the previous one.

The position of an element in the array is determined by its index.

$$\text{Address} = \text{base_Address} + i \times \text{element_size}$$

The memory layout of arrays has several implications for performance.

1. Cache locality:

Arrays have good cache locality because elements that are close to each other in the array are also close to each other in memory.

2. Efficient access time:

Since elements are stored contiguously, accessing an element by its index is efficient.

The compiler can calculate the memory address directly, and there is no need for pointer arithmetic or additional calculations.

3 sequential access: Iterating over an array is efficient due to the sequential layout. This is beneficial for algorithms that involve sequential processing of data.

3.) How is a two-dimensional array declared in C?

- a) `int array[2];`
- b) `int array [2][2];`
- c) `int array [2];`
- d) `int array [][2];`

4.) All arrays in C must have a fixed size that is determined at compile time. (True or False)

5.) What does the expression `arr[3]` represent in C, assuming `arr` is an array?

- a. The value at index 3
- b. The memory address of index 3
- c. the size of the array
- a. The sum of all elements up to index 3

~~12~~
12/12/23